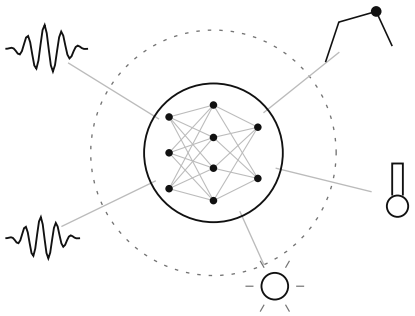
A Local Decision Model for Physical Projects



many observations, one local model

ardchain

Independent concept paper / 8 October 2026

Ready-trained local inference for ESP32-class devices

Original theoretical proposal; no experimental results

Abstract

This paper proposes a ready-trained local decision system for small physical projects. A user connects supported sensors, defines the states or events that matter, and maps structured decisions to application behavior. Inference may share an ESP32-class controller with the application or run on a companion module when resource demands justify it. The product goal is to make useful local intelligence reusable across projects without requiring each user to train a model. The interface is inspired by Jev's state-and-question pattern; the proposal does not assume access to Jev weights or equivalent capability on a microcontroller. We distinguish established embedded inference components from a proposed task-configurable decision core and the open research goal of arbitrary task descriptions. The document specifies architecture, resource admission, user workflows, adoption hypotheses and an evaluation plan. It reports no hardware experiments or measured performance.

1. Motivation and scope

A builder should be able to add perception and bounded judgment to an existing thing without turning the project into a machine-learning exercise. The intended unit of value is a useful behavior: a sound-aware indicator, a responsive toy, or a device that interprets several sensor readings together. User training is deferred; compact runtime behavior and clear limits are central to the proposal.

2. Product contract and task configuration

The product is conceived as one local intelligence interface that can serve many applications. Its implementation may contain several specialized neural components. A common interface does not imply that one set of weights understands every sensor, environment and task. The first requirement is that users can discover what the installed system actually supports.

2.1 State, question, answer

Jev evaluates a supplied state against typed questions and returns structured values that software can consume [1]. We adopt that interaction pattern as a design inspiration. For a physical device, the state would contain observations, timestamps, sensor health and a short history. A question defines the permitted answers. The response contains an answer, a validity status and optional model scores. The application decides whether to act; unknown and unavailable are explicit outcomes.

For example, a desk device could ask whether a supported spoken command was detected while its enable button was held. Its answers could be command_detected, no_command and unknown. A threshold and a short time window determine when to report the event. The configuration also defines which light output may change. This separates recognition from permission to act.

2.2 What the user configures

Table 1. Task configuration fields.

Configuration	Meaning
Inputs	Choose supported microphone or sensor adapters, units and sampling settings.
Question	Define the relevant supported states and the allowed output values.
Context	Add time windows, button state, history or additional valid sensor readings.
Response	Map valid decisions to prepared actions; specify what happens on unknown.
Resource limit	Select a profile that fits alongside the existing application.

2.3 Task descriptions as an open question

A user could eventually describe the goal in ordinary language. Initially, a guided form can translate supported intentions into a checked task configuration. That translation can run during setup on a local computer. The device then executes the resulting configuration offline. Arbitrary text-conditioned neural judgment on ESP32 remains a separate research objective; this paper does not assume it has been solved.

Defining a new label does not teach the model a new sound or physical phenomenon. The interface must reject unsupported labels or explain which additional pretrained capability would be required. Configuration changes may alter thresholds and actions, while model capabilities remain bounded by the installed release.

3. Proposed system architecture

The runtime separates the physical signal, its interpretation and the application response. This makes the system easier to adapt and lets developers replace one stage without changing every project. Figure 1 describes a proposed data path rather than a tested implementation.

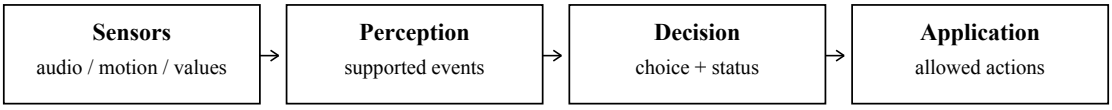


Figure 1. Proposed local data path. Sensor adapters produce observations; perception produces supported events; a decision layer returns typed outcomes; the application applies its own action policy.

3.1 Sensor adapters and perception

Adapters handle acquisition, units, timestamps and missing data. An audio path also requires buffering and feature processing. Perception components translate those inputs into supported events or representations. Spoken command recognition and environmental sound classification are distinct capabilities. Espressif documents offline command recognition on ESP32-S3, including a configurable English or Chinese command set [2]; this does not establish recognition of arbitrary sounds.

3.2 A bounded decision core

The proposed core combines prepared observations with a task configuration and a small history. It may choose among supported states or produce a bounded score. Candidate implementations include a compact learned classifier over structured features, explicit rules, or a combination. A learned core is useful only if evaluation shows a meaningful gain over simple rules. No particular network architecture or model size is selected in this paper.

An unknown response can arise from uncertain perception, a missing sensor, conflicting observations or a missed deadline. Numeric scores should be described as model scores until probability calibration is demonstrated for the deployed task. A high score alone does not establish correctness in a new environment.

3.3 Runtime and project interface

The same logical interface should work in both deployment modes: submit observations and a question identifier, then receive a timestamped decision. Streaming tasks can publish events asynchronously. The contract must include task version, input validity and staleness so the host can ignore an answer that no longer applies. A setup tool installs compatible models and configuration; ongoing inference requires no cloud request.

4. Deployment within a project budget

The preferred mode is whichever meets the needs of the physical project with the least unnecessary hardware. Installing on the main controller is appropriate when the application and inference fit together. A companion processor is an option when isolation, memory or throughput warrants it. Neither mode is assumed to be universally better.

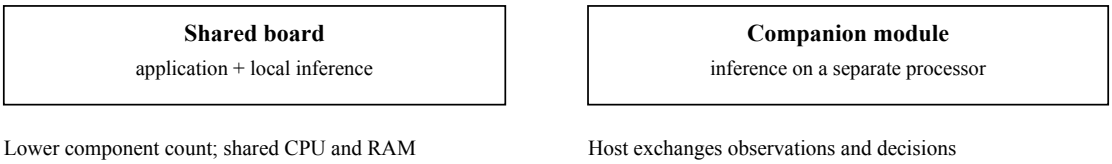


Figure 2. Two execution modes, one task interface. The companion mode moves inference compute off the host but still requires communication, power and integration.

4.1 Storage, working memory and compute

Model storage, working memory and processor time are separate budgets. A dedicated flash partition or external storage can hold weights and configuration. PSRAM can provide working memory on compatible hardware, with allocation and access restrictions documented by Espressif [4]. Neither creates an independent processor. Extra storage therefore cannot guarantee that the main application retains its timing or memory headroom.

A board profile must cover the complete deployed graph: acquisition buffers, features, tensors, intermediate activations, task stacks and application allocations. Installing only the components needed by the task can reduce demand. Event-triggered inference may save work in some scenarios, although a sound detector may still need continuous capture or an inexpensive always-active gate.

4.2 Admission and scheduling

Let R_total denote usable working memory, R_app the application peak, R_ai the inference peak and R_margin a reserved allowance. Shared-board admission requires R_app + R_ai + R_margin <= R_total, with internal RAM and PSRAM checked separately. An analogous storage check covers firmware, models and update space. These are planning inequalities, not performance predictions.

Timing admission also requires measured end-to-end behavior under simultaneous application load. The runtime should bound work frequency, queue length and input age. A low task priority alone cannot guarantee that inference will never delay the application. If an input is dropped or a result becomes stale, the answer should explicitly say unavailable rather than silently reuse an old decision.

ESP-DL supplies deployment and quantization tools for supported Espressif targets [3]. Portability still requires compatible operators, preprocessing and hardware tests. Arduino boards vary widely; Raspberry Pi integration would use a target-appropriate runtime. A shared task format is a portability goal, not a universal binary.

5. Application scenarios

The following scenarios illustrate how the interface could be reused. They are design examples, not measured demonstrations. A scenario is eligible only when the chosen model, sensor combination and environment are validated together.

5.1 A local sound-aware desk device

A microphone feeds a supported event detector. The user selects an available event and maps it to an indicator, counter or local notification. A cooldown avoids repeated outputs for one event. The setup tool shows live detections and asks the user to try both desired and distracting inputs before installation. Recognizing a particular custom doorbell or machine sound may require new data and model work; naming it in a form is insufficient.

5.2 A responsive toy or physical interface

A prepared speech-command recognizer or supported motion detector selects a small set of interactions. A toy may light up or change a low-power behavior after a valid command. The same recognition can be reused for a desk controller, a classroom exercise or a hands-free indicator by changing the permitted actions. This freedom comes from recombining known capabilities rather than assuming unlimited understanding.

5.3 Several sensors, one interpretable state

A workbench could combine vibration, temperature and an enable signal to report a supported operating state. The interface records units and timing so the decision layer does not compare incompatible or stale values. A simple rule-based baseline should be tested first. A learned decision component is justified only when it improves robustness or captures useful patterns the baseline misses. Broad fault diagnosis across unrelated machines is beyond the evidence available here.

5.4 Keeping the action proportionate

Early projects should make results observable: a light, event log or local notification lets a person see and correct errors. The response policy belongs to the application. The intelligence component should not receive unrestricted control simply because it produced a confident label. For a stale or unsupported observation, the configured default can preserve the existing state or notify the user.

5.5 Why local operation matters

Local inference can keep raw inputs on the device and avoid dependence on a remote inference service. A recipe that sends events to another system should say so explicitly. Offline operation still depends on power, valid sensors and a compatible installed model. It does not by itself guarantee low energy use, low latency or reliability; those depend on the complete implementation.

6. A path toward broader adoption

The mass-adoption hypothesis is that useful, repeatable projects matter more to most people than selecting a neural architecture. The platform should lead with an outcome and make the supported hardware and capability visible. This section proposes a product path, not a market forecast.

6.1 First success without model training

Begin with one supported ESP32-S3 configuration and one understandable experience. A prepared voice or event-driven indicator is a candidate because its output is easy to observe. Provide the exact hardware list, prepared connections and a local setup flow. The user chooses a supported event, assigns a response, checks it on live input and installs. They should not need to manage tensors, convert models or inspect a build log to complete that path.

6.2 Share complete project recipes

A useful recipe includes the board variant, sensor wiring, capability version, task configuration and permitted actions. The installer checks these against the connected hardware. People can then change the application behavior while keeping a working perception component. Recipes should preserve their dependency versions so a shared project remains reproducible after the catalog changes.

A broad community could emerge through makers, educators and module designers before the product reaches less technical users. That progression needs evidence: watch whether people complete a setup, build a second project and successfully reproduce someone else's recipe. Downloads and curiosity alone do not establish sustained usefulness.

6.3 Resource fit as part of the interface

The setup tool should explain whether a task fits alongside the existing application. A profile can display known storage and RAM requirements, the tested execution schedule and unresolved measurements. Estimates must be labeled as estimates. If the application has no profile, compatibility should remain unverified until the combined workload is tested. A clear recommendation to reduce a task or use a companion module is more useful than a generic compatibility badge.

6.4 What would make this product distinct

Embedded inference libraries already provide important building blocks [2,3,5]. The proposed contribution is their integration into a consistent state-and-question interface, a checked installation path and reusable recipes with resource budgets. Its advantage must be demonstrated by easier setup and useful behavior across multiple projects. A new label or interface alone would not establish a technical advance.

Pricing, hardware bundling and distribution remain open. Start by testing the workflow on available components; decide whether an assembled kit is needed from observed setup failures. User-supplied training can be added later without making it a prerequisite for the first useful experience.

7. Evaluation plan and research questions

The concept needs evidence at both model and product levels. The proposed evaluation compares simple rules, a perception-only system and the full decision pipeline on the same tasks. Every result should identify the exact board, model release, sampling settings, application workload and test conditions. No results from these experiments are available yet.

Table 2. Proposed evaluation questions and evidence.

Question	Evidence to collect
Does it recognize the event?	Missed events and false detections on held-out sessions, users and environments; include confusing non-target inputs.
Does uncertainty help?	Compare accept/reject behavior; measure accepted errors and coverage. Test calibration before calling scores probabilities.
Does it fit the project?	Peak internal RAM and PSRAM, storage, power and end-to-end response time under concurrent application load.
Does the host keep working?	Deadline misses, dropped inputs, watchdog behavior, queue limits and stale-result handling.
Can a new user use it?	Observed setup completion, intervention needed, error recovery and successful reuse of a shared recipe.
Does the learned core add value?	Compare with explicit rules on the same held-out inputs, including maintenance and integration complexity.

7.1 Generalization without user training

The central research question is how much task variation a ready-trained compact system can support without collecting new user data. Evaluate tasks outside the preparation set, not merely different labels for the same recordings. Report which changes are configuration-only, which need a new pretrained component, and which exceed the supported domain. Reusing one interface across applications should not be reported as neural generalization.

7.2 Words as task configuration

A further experiment can compare guided forms with a local setup assistant that turns a text goal into a validated configuration. Measure unsupported-request detection and whether users understand the generated task. This assesses interface usefulness. Demonstrating arbitrary new tasks from text at runtime would require a separate model benchmark, resource profile and evaluation of semantic failures.

A staged evaluation should prioritize one end-to-end working task, then a second application using the same interface, followed by controlled expansion. Publish failures alongside successes so the next release extends demonstrated capability rather than only broadening the promise.

8. Proposed development sequence

Stage 1: establish the contract. Define observation schemas, supported outcomes, input health, unknown responses and the resource-profile format. Select one board and one prepared sensor path. This can be developed before committing to a particular decision-network architecture.

Stage 2: prove a complete local loop. Deploy a ready-trained perception component, compare simple rules with a compact learned decision candidate, and connect the result to a visible output. Profile the combined workload. A live demonstration should show operation with the remote inference path absent.

Stage 3: test reuse. Build a second project through the same interface and test recipe installation with new users. Extend hardware or sensing support only after reproducing the behavior on the additional configuration. Preserve the companion-module option for tasks that exceed a shared-board budget.

Stage 4: investigate wider task freedom. Explore constrained text-based setup and broader prepared representations. User training and arbitrary language-conditioned decision making remain future work, with separate evidence requirements.

9. Conclusion

A ready-trained local intelligence layer could become a reusable part of physical projects if it offers supported perception, typed decisions and predictable resource use through an understandable interface. The immediate engineering opportunity is a bounded system people can configure and reproduce. The wider ambition is a compact model adaptable to more tasks without user training. This paper states that ambition and a way to investigate it; it does not claim that a universal embedded decision model already exists.

References

[1] TypeSafe AI. Introduction. Jev state and typed-question interface. Read primary documentation. Accessed 8 October 2026.

[2] Espressif Systems. ESP-SR: MultiNet command recognition on ESP32-S3. Read primary documentation. Accessed 8 October 2026.

[3] Espressif Systems. ESP-DL User Guide: Getting Started. Read primary documentation. Accessed 8 October 2026.

[4] Espressif Systems. ESP-IDF: Support for External RAM, ESP32-S3. Read primary documentation. Accessed 8 October 2026.

[5] Google AI Edge. LiteRT for Microcontrollers: Overview. Read primary documentation. Accessed 8 October 2026.